

AI in Cybersecurity and Cloud Computing

Lesson 3

Security Data Processing: From Logs to a Ready Dataset

Dr. Mohammed Tawfik
Head of the Department of Cyber Security and Cloud Computing
Faculty of Information Technology, Ajloun National University
kmkhol01@gmail.com

Learning Objective

- Explain the eight steps that turn raw security data into a ready dataset, apply them to security cases, and describe how AI makes each step easier.

Key Concepts

- Why data processing matters
- Collect, parse and normalise logs
- Clean the data; outliers may be attacks
- Build features in time windows
- Turn data into numbers; add labels
- Split by time; avoid data leakage
- Cases: e-mail, network traffic, malware, payments
- How AI makes each step easier

Recap

- AI is used against phishing, malware, intrusions, account takeover, fraud and more
- Every case starts with data, mostly logs
- Five processing steps: collect, parse, clean, turn into numbers, split

Today: each step explained, with examples, cases and how AI helps

Why Data Processing Matters

Definition **Data processing:** the steps that turn raw security data into a clean, labelled table of numbers that a model can learn from.

Explanation

A model only learns what is in its data: “garbage in, garbage out”. Raw logs contain errors, copies, mixed formats and noise. Processing removes these problems and keeps the clues that show an attack.

```
Nov 03 02:14:55 srv1 sshd[4122]: Failed …
2026-11-02T23:28:01Z EventID=4625 …
DENY TCP 203.0.113.45:51600 -> …
```

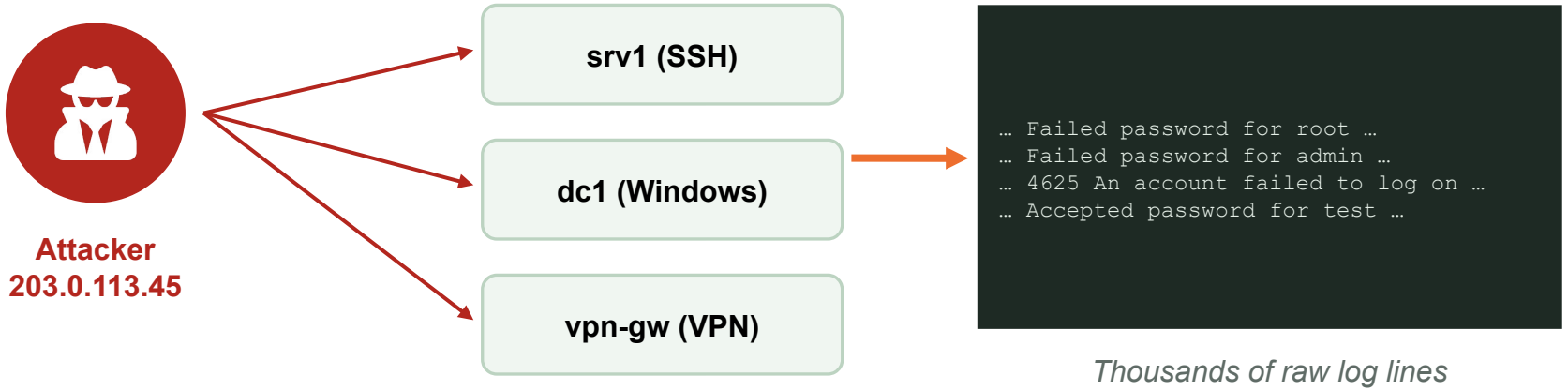


Failed	Users	Night	Label
120	35	1	attack
1	1	0	normal



How AI helps: AI tools automate many processing steps today, but the analyst decides which logs, features and labels make sense.

Our Story: Password Guessing on University Servers



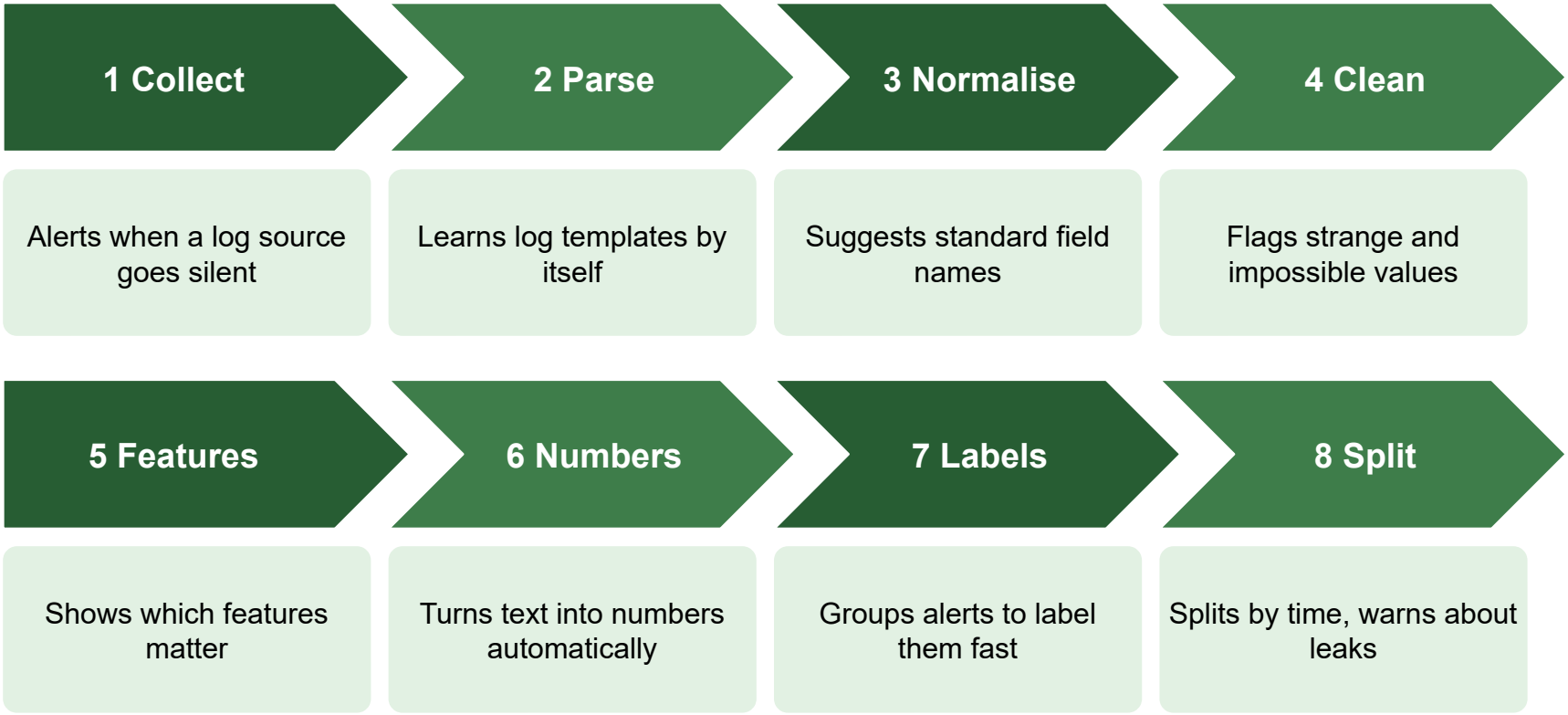
Thousands of raw log lines

Definition **Brute-force attack:** trying many passwords, one after another, until one works.

Goal: an AI detector for password guessing → first we need clean data

All names and addresses are invented

Eight Steps, and Where AI Helps



Green boxes = how AI helps in each step

Step 1: Collect the Right Logs

Definition **Log collection:** sending the logs of many systems to one central place, usually the SIEM.

Explanation

First decide which systems record what the detector needs. For password guessing that is every place where people log in. A small program (agent) on each server sends its logs to the SIEM. All server clocks must be synchronised (NTP), or events cannot be put in the right order.

Example

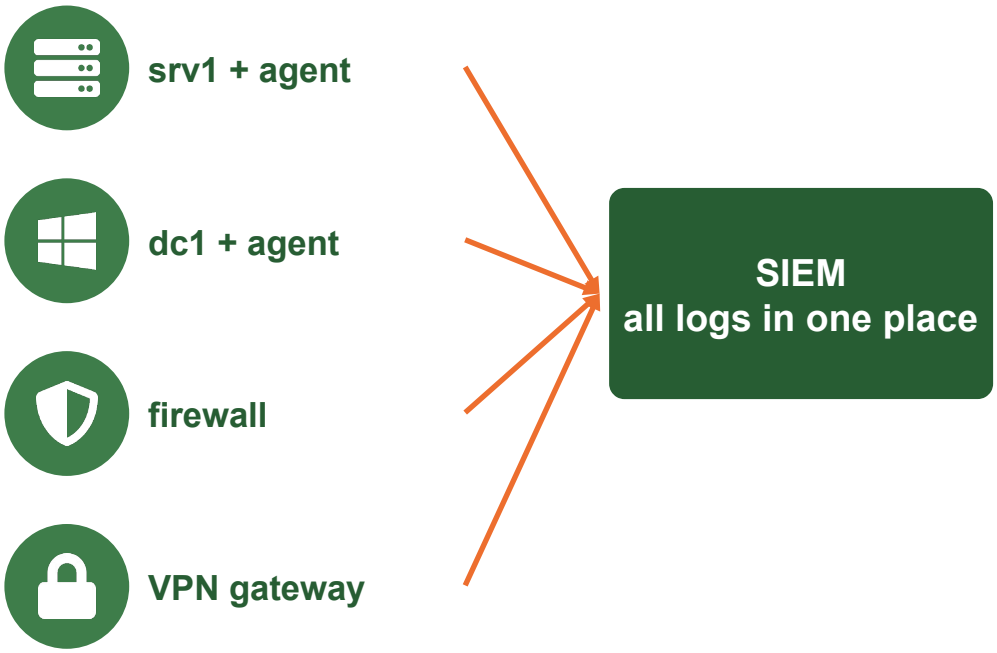
Log source	Records
Linux auth log	SSH: Failed / Accepted password
Windows Security	4625 failed, 4624 success
VPN gateway	remote logins + country
Firewall	connections to ports 22, 3389



How AI helps: SIEM machine-learning features alert when a log source suddenly goes silent or its volume jumps. Attackers often switch logging off, so a silent source is itself a warning.

Collect | Parse | Normalise | Clean | Features | Numbers | Labels | Split

Example: Collecting Logs in Practice



Timeline with a wrong clock

Server time	Event
02:21 (srv1, 7 min late)	Accepted password
02:28 (dc1)	Failed logon
02:28 (dc1)	Failed logon

The success looks earlier than the failures: the attack pattern is hidden

Rule: every server synchronises its clock (NTP) before logs are collected

Step 2: Parse Each Line into Fields

Definition **Parsing:** splitting a log line into named fields.

Explanation

A log line is one long text. Parsing cuts it into pieces with names: time, server, program, result, user, address, port. After parsing, each line is one row of a table and each piece is one column. The SIEM parses common formats automatically; unusual logs need a parser written by the team.

Example

```
Nov 03 02:14:55 srv1 sshd[4122]:
Failed password for root
from 203.0.113.45 port 51522 ssh2
```



time	host	result	user	source_ip
02:14:55	srv1	failed	root	203.0.113.45



How AI helps: log-parsing algorithms (for example Drain) read millions of lines and learn the templates by themselves. Generative AI can write a parser for a new log format from a few example lines; the analyst tests it before use.

Collect | **Parse** | Normalise | Clean | Features | Numbers | Labels | Split

Example: Templates Learned from the Logs

```
Failed password for root  from 203.0.113.45 port 51522 ssh2
Failed password for admin from 203.0.113.45 port 51530 ssh2
Failed password for test  from 198.51.100.9 port 40011 ssh2
Accepted password for test from 203.0.113.45 port 51601 ssh2
```



Template A: Failed password for <*> from <*> port <*> ssh2

Template B: Accepted password for <*> from <*> port <*> ssh2

Definition **Log template:** the fixed part of a log message; the changing parts are its variables.

Variables <*> become columns: user, source_ip, port

Step 3: Normalise Names, Formats and Time

Definition **Normalising:** giving the same thing the same name, format and meaning in every log source.

Explanation

Each system names things differently: SSH writes “from”, Windows writes “SourceIP”, the firewall writes “src”. Normalising gives one name (source_ip), one format and one meaning to each field. Time is converted to one time zone, UTC, so events from different servers can be put in order.

Example

Source	Before	After
SSH	from=…	source_ip
Windows	SourceIP=…	source_ip
Firewall	src=…:51600	source_ip
SSH	Failed password	failed_login
Windows	EventID 4625	failed_login



How AI helps: common schemas such as the Elastic Common Schema (ECS) and OCSF give standard field names. AI assistants in the SIEM suggest which original field matches which standard field, and the analyst confirms.

Collect | Parse | **Normalise** | Clean | Features | Numbers | Labels | Split

Example: Putting the Attack in Order (UTC)

Before

Raw time	Source	Event
02:28:01 (UTC+3)	SSH	failed
23:28:05 Z	Windows	failed
23:27:50 Z	Firewall	scan
02:28:40 (UTC+3)	SSH	success

After

UTC time	Source	Event
23:27:50	Firewall	scan
23:28:01	SSH	failed
23:28:05	Windows	failed
23:28:40	SSH	success

One time zone (UTC) → the attack appears as one sequence in one minute

Z = UTC · Amman is UTC+3

Step 4: Clean the Data

Definition **Data cleaning:** finding and fixing errors, gaps and noise in the data.

Explanation

Real logs are messy: empty fields, the same event sent twice, lines cut by a crash, impossible values and routine noise such as a monitoring account that logs in every 10 seconds. Fix what is clearly an error, mark what is unknown, and never delete something only because it looks strange.

Example

Problem	Example
Missing	country of a VPN login empty
Duplicate	same event sent twice
Impossible	port 70000
Noise	health check every 10 s



How AI helps: AI flags strange and impossible values automatically, finds near-duplicate events, and can estimate a missing value from the other columns. The analyst checks before anything is deleted.

Collect | Parse | Normalise | **Clean** | Features | Numbers | Labels | Split

Example: Fixing Gaps, Copies and Errors

Before

#	user	country	port
1	sara		443
2	omar	JO	22
3	omar	JO	22
4	root	NL	70000

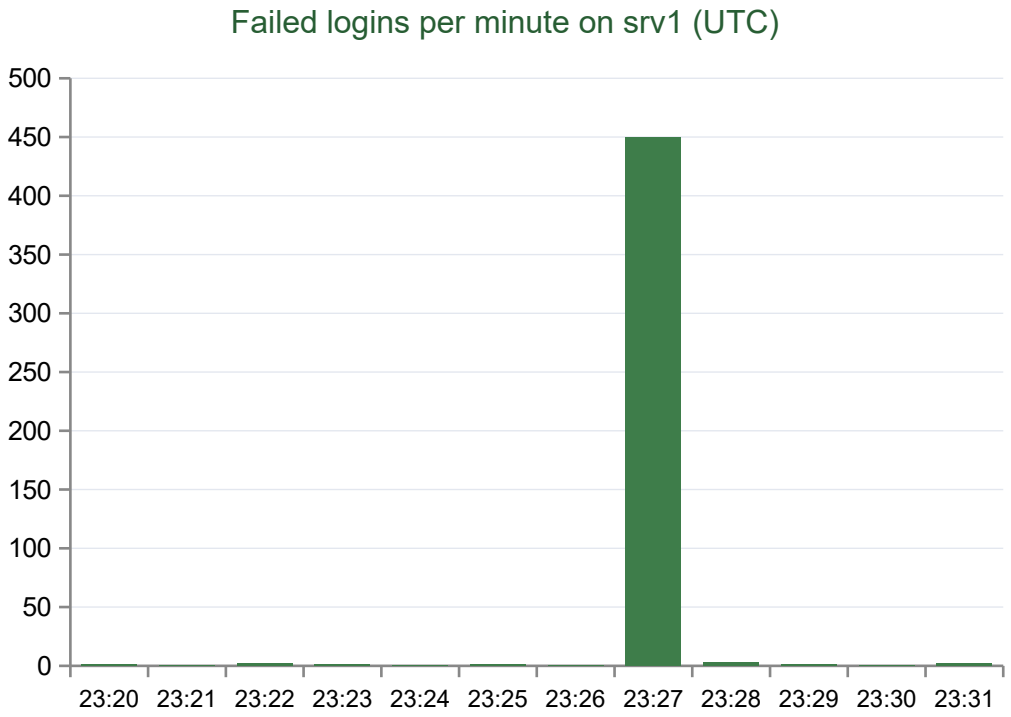
After

#	user	country	port
1	sara	unknown	443
2	omar	JO	22
3	(duplicate removed)		
4	(impossible: removed)		

- Missing → fill or mark “unknown”
- Duplicate → keep one copy
- Impossible or broken → repair or remove
- Routine noise → filter it out

Example: Outlier, Error or Attack?

Definition **Outlier:** a value very far from the other values.



Logging error?
→ fix it

Real event?
→ keep it: it is the attack

Never delete outliers blindly

Step 5: Build Features

Definition **Feature:** one measurable property of an example that may help the model decide.

Explanation

One failed login says almost nothing; a pattern over a few minutes says a lot. So we group the events of each address in a time window (for example 5 minutes) and count: failed logins, different user names tried, a success after failures, the hour. Each row now describes one address in one window.

Example

Window	Source IP	Failed	Users	Success
23:25–23:30	203.0.113.45	120	35	yes
23:25–23:30	10.0.0.21	1	1	no
23:30–23:35	10.0.0.34	0	1	no

Time window = a fixed period in which events are grouped and counted



How AI helps: after a first model is trained, it shows which features it relied on most (feature importance), so the team keeps the useful ones. Deep learning can even learn features directly from raw text.

Collect | Parse | Normalise | Clean | **Features** | Numbers | Labels | Split

Example: Good and Bad Features

Feature	Good or bad?	Why
Failed logins in 5 minutes	Good	Guessing tools try many passwords quickly
Different user names tried	Good	Attackers try root, admin, test, …
Success after many failures	Good	The attacker may have guessed one
New address or country	Good	Staff log in from known places
Log line number	Bad	Carries no clue
“Blocked by analyst”	Bad	Known only after the attack (leakage)

Step 6: Turn Everything into Numbers

Definition **Encoding:** turning text, categories and yes/no values into numbers a model can use.

Explanation

Models calculate, so every column must be a number. Yes/no becomes 1/0. A time becomes the hour and a night flag. A category such as the country gets one 1/0 column per value (one-hot encoding), so no false order appears. Some models also need all numbers on the same scale (scaling).

Example

Value	Becomes
success: yes	1
time 23:12	hour 23, night 1
country NL	JO 0 · NL 1 · US 0

In Orange: Preprocess → Continuize (one-hot) and Normalize (scaling)

How AI helps: tools such as Orange encode and scale all columns automatically. Language models turn free text, such as the body of an e-mail or a log message, into numbers that keep its meaning.

Collect | Parse | Normalise | Clean | Features | **Numbers** | Labels | Split

Example: One-Hot Encoding

Definition **One-hot encoding:** giving each category its own yes/no (1/0) column.

Before

Window	Country
1	JO
2	NL
3	JO

After

Window	JO	NL	US
1	1	0	0
2	0	1	0
3	1	0	0

JO = 1, NL = 2, US = 3 would create a false order

In Orange: Preprocess → Continuize

Step 7: Add the Labels

Definition **Label:** the correct answer for an example, here attack or normal.

Explanation

A supervised detector needs the correct answer for each row. In a SOC the labels come from analysts who closed past alerts as real or false, from threat-intelligence lists of known bad addresses, and from red-team exercises in which the university attacks itself on purpose and records when.

Example

Source IP	Failed	Users	Label	From
203.0.113.45	120	35	attack	analyst
10.0.0.21	1	1	normal	analyst
198.51.100.66	40	9	attack	red team

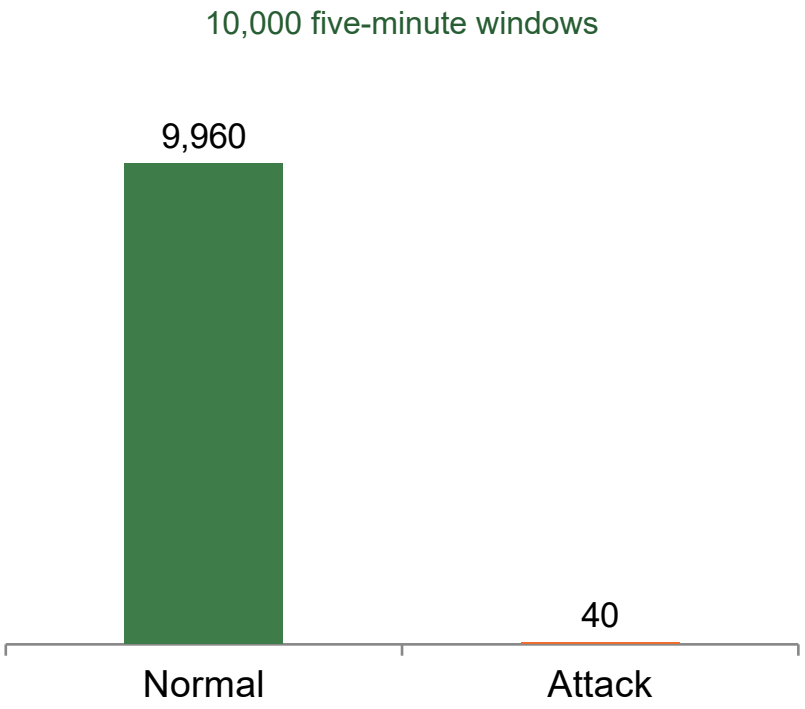


How AI helps: clustering groups similar alerts so the analyst labels a whole group at once. The model can also ask the analyst only about the cases it is unsure of (active learning), which saves many hours of labelling.

Collect | Parse | Normalise | Clean | Features | Numbers | **Labels** | Split

Example: Attacks Are Rare

Definition **Imbalanced data:** data in which one class, such as attacks, is very rare.



- Attacks: 40 of 10,000 (0.4%)
- “Always normal” is 99.6% correct
- And it catches no attack
- Collect more attack examples
- Judge by attacks caught

Step 8: Split the Data

Definition **Training set / test set:** the examples the model learns from / the examples kept aside to check it.

Explanation

Before training, the table is split. The model learns from the training set (about 70%); the test set (about 30%) is a final exam it never sees. In security we split by time: train on older weeks, test on the newest, because attackers change and the detector will always meet new data.

Example



How AI helps: ML tools split by time and keep the share of attacks the same in each part automatically. Some tools also warn about columns that look too much like the answer, a sign of possible data leakage.

Collect | Parse | Normalise | Clean | Features | Numbers | Labels | **Split**

Example: Data Leakage

Definition **Data leakage:** information in the training data that will not be available when the model is used for real.

Column filled later

"blocked_by_analyst" is written after the attack is known

Same event in both sets

A duplicate lands in training and in test

Answer hidden in a feature

"on_blacklist" built from the same incident reports as the labels

Ask of every column: will I have this value when the attack happens?

The Same Steps, Other Security Cases

Case	Raw data	Example features after processing
Password guessing	SSH, Windows, VPN logs	failed logins, users tried, success after failures
Phishing e-mail	E-mail gateway logs + message	links, domain age, urgent words, sender mismatch
Network attack	Network packets	duration, bytes, connections per second, failed share
Malware	The file itself	size, signature, imported functions, packed or not
Card fraud	Bank payment records	amount ÷ usual, new country, night, payments per hour

Same eight steps · different raw data · different features

Case: A Phishing E-mail Becomes Features

Raw data

E-mail gateway

From: IT Support <it-support@ajloun-nationa1.com>
Reply-To: helpdesk@mail-box.co
Subject: URGENT: password expires today

Click here within 24 hours: http://203.0.113.45/login
Attachment: form.zip

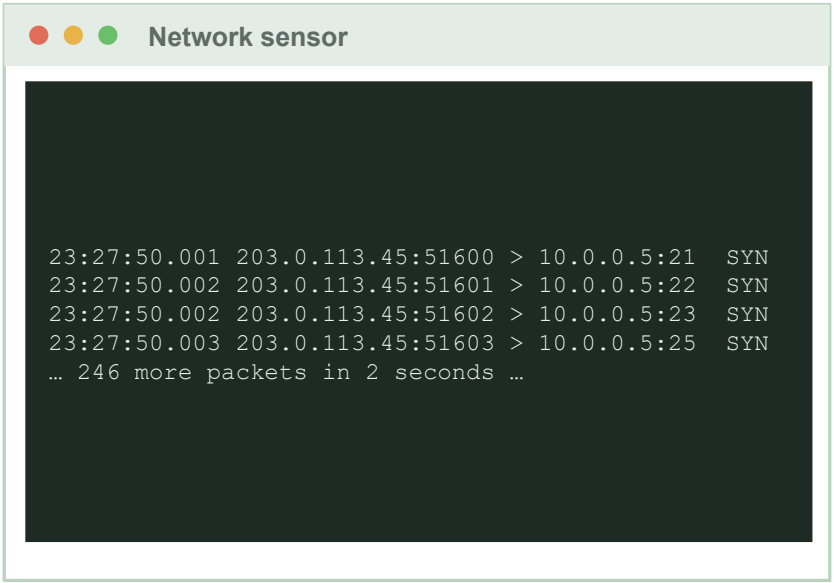
Features (after processing)

Feature	Value
Links in body	1
Link is an IP address	1
Sender domain age (days)	3
Reply-To ≠ From	1
Urgent words	2
Zip attachment	1

How AI helps: language models read the whole message and turn it into numbers that keep its meaning, so new phishing wording is still recognised; parsing and normalising of headers is automatic in mail gateways.

Case: Network Traffic Becomes Flows

Raw data



Features (after processing)

Flow feature	Value
Duration	2 s
Different ports contacted	250
Bytes sent	almost 0
Failed connections	98%
Label	attack (port scan)

How AI helps: flow tools such as Zeek and CICFlowMeter turn millions of packets into flows with ready features automatically; the AI-based IDS then learns from those flows.

Case: A File Becomes Malware Features

Raw data

Sandbox report

File: invoice.pdf.exe (312 KB)
Type: Windows program
Digital signature: none
Imports: keyboard capture, network
Packed: yes
In sandbox: connects to 198.51.100.7, copies itself to Startup

Features (after processing)

Feature	Value
Size (KB)	312
Double extension	1
Signed	0
Keyboard-capture import	1
Packed	1
Starts at boot	1

How AI helps: sandboxes record behaviour automatically, and models trained on millions of files decide which of these features matter most; analysts only review the doubtful files.

Case: A Card Payment Becomes Fraud Features

Raw data

Bank payment record

Field	Value
Card	•••• 4417
Amount	1,900 JD
Shop	electronics, abroad
Time	03:40
Card present	no (online)

Features (after processing)

Feature	Value
Amount ÷ usual amount	47
New country for card	1
Night (00–05)	1
Payments in last hour	6
Card present	0



How AI helps: fraud systems compute these features for every payment in milliseconds and combine a model trained on past frauds with anomaly detection for new fraud patterns.

Worked Example: One Log Line, All Eight Steps

1 Collect	srv1 agent → SIEM, clock synchronised
2 Parse	time · srv1 · Failed password · root · 203.0.113.45
3 Normalise	2026-11-02 23:14:55 UTC · failed_login · user=root
4 Clean	not a duplicate, not broken: keep
5 Features	window 23:10–23:15: failed 120 · users 35 · success yes
6 Numbers	120 · 35 · 1 · hour 23 · night 1 · NL = 1
7 Label	attack (confirmed by the analyst)
8 Split	November → test set

The Ready Dataset

Failed	Users tried	Success after	Hour	Night	JO	NL	US	Label
120	35	1	23	1	0	1	0	attack
1	1	0	9	0	1	0	0	normal
0	1	0	10	0	1	0	0	normal
8	2	0	23	1	0	0	1	normal
260	50	0	2	1	0	0	1	attack

Clean · labelled · all numbers · split by time

How AI Makes Data Processing Easier

Step	By hand	With AI help
1 Collect	Check every source works	Alert when a source goes silent
2 Parse	Write a pattern for every format	Learn templates; draft parsers
3 Normalise	Map every field by hand	Suggest the standard field names
4 Clean	Read rows to find errors	Flag strange values and near-copies
5 Features	Guess which clues matter	Show feature importance
6 Numbers	Convert each column	Encode and scale automatically; text → numbers
7 Labels	Label every row	Group similar alerts; ask only unsure cases
8 Split	Split and check by hand	Split by time; warn about leakage

The analyst checks and approves; AI does the heavy work

Practice Data

Loghub: OpenSSH logs (GitHub)
2,000 real SSH log lines, raw and parsed, with learned templates: steps 1–5
https://github.com/logpai/loghub/tree/master/OpenSSH

Network Intrusion Detection (Kaggle)
25,192 connections, 41 features incl. categories protocol_type, service, flag: steps 6–8
https://www.kaggle.com/datasets/sampadab17/network-intrusion-detection

Real data, no passwords, safe to open

Lab: Processing Security Data in Orange

Part A: the SSH log (steps 2–5)



Part B: the intrusion dataset (steps 6–8)



- How many failed logins are in the SSH file? Which user names were tried?
- How many columns before and after one-hot encoding? Why?

Hand in: screenshots of both workflows + your answers

Class Activity: Process These Lines

SSH log of srv1 (Amman time, UTC+3):

```
1  Nov 03 02:28:01 srv1 sshd: Failed password for root from 203.0.113.45
2  Nov 03 02:28:09 srv1 sshd: Failed password for admin from 203.0.113.45
3  Nov 03 02:28:09 srv1 sshd: Failed password for admin from 203.0.113.45
4  Nov 03 02:28:20 srv1 sshd: Failed password for test from 203.0.113.45
5  Nov 03 02:28:41 srv1 sshd: Accepted password for test from 203.0.113.45
```

- Write the times in UTC and find the problem in the lines
- Build one feature row: failed logins, users tried, success after failures, night
- Which label would you give? Why?

Key Terms

Term	Definition
Data processing	Steps from raw security data to a clean, labelled table of numbers
Log collection	Sending all logs to one central place (SIEM)
Parsing / log template	Splitting a line into fields / the fixed part of a message
Normalising	Same name, format and meaning in every source; one time zone (UTC)
Data cleaning	Finding and fixing errors, gaps and noise
Outlier	A value very far from the others: error or attack
Feature / time window	Measurable property / fixed period for counting
Encoding / one-hot	Turning values into numbers / one 1-0 column per category
Label / imbalanced data	The correct answer / one class is very rare
Training / test set	Examples to learn from / examples kept for the final check
Data leakage	Training data contains what will not exist in real use

Review Questions

- Name the eight steps of data processing in order.
- Why must all server clocks be synchronised before collecting logs?
- What is a log template? Give one from SSH logs.
- Convert 02:14 Amman time (UTC+3) to UTC. Why do we normalise time zones?
- Why should a security analyst not delete outliers blindly?
- Name two ways AI makes labelling easier.
- Why do we split security data by time? Give one example of data leakage.

Summary

- Eight steps: collect, parse, normalise, clean, features, numbers, labels, split
- Normalise to one time zone (UTC); never delete outliers blindly
- Features count events in time windows; everything becomes numbers
- Labels come from analysts, threat intelligence and red teams; attacks are rare
- Split by time and avoid data leakage
- The same steps work for e-mail, network, malware and fraud data
- AI does much of the work; the analyst checks and approves

Challenge Question (Optional)

For students who want to go further

The SOC builds a brute-force detector. Its test score is 99.9%. Later you discover that the table contains a column “blocked_by_firewall”, filled in a few seconds after each attack, and that the training and test sets were mixed from all months.

What went wrong, and how would you fix it?